UNIT – 1

ASSESSMENT – 1

PYTHON CODE

1.Water Jug Problem

```
from collections import deque

def water_jug_bfs(jug1_cap, jug2_cap, target):
    start = (0, 0)
    visited = set([start])
    queue = deque([(start, [start])])

    while queue:
        (x, y), path = queue.popleft()

        if x == target or y == target:
            return path

        next_states = [
            (jug1_cap, y),         # Fill jug1
            (x, jug2_cap),         # Fill jug2
            (0, y),                # Empty jug1
            (x, 0),                # Empty jug2
```

```
            (x - min(x, jug2_cap - y), y + min(x, jug2_cap - y)),  # Pour jug1 ->
jug2
            (x + min(y, jug1_cap - x), y - min(y, jug1_cap - x))   # Pour jug2 ->
jug1
        ]

        for state in next_states:
            if state not in visited:
                visited.add(state)
                queue.append((state, path + [state]))

    return None

# Solve: 4-gallon and 3-gallon jugs, target = 2 gallons
solution = water_jug_bfs(4, 3, 2)
print("Water Jug Solution Path (jug1, jug2):")
for step, state in enumerate(solution):
    print(f"Step {step}: {state}")
```

2. Mars Rover Intelligent Agent

```
import random

class MarsRoverAgent:
    def __init__(self):
        self.position = (0, 0)
```

```
    self.battery = 100
    self.samples_collected = 0
    self.internal_map = {}  # model of the world (model-based agent)

def perceive(self):
    percept = {
        "terrain": random.choice(["rocky", "sandy", "flat"]),
        "obstacle_ahead": random.choice([True, False]),
        "sample_detected": random.choice([True, False]),
        "battery_level": self.battery
    }
    return percept

def update_model(self, percept):
    self.internal_map[self.position] = percept["terrain"]

def decide_action(self, percept):
    if percept["battery_level"] < 20:
        return "return_to_charge"
    if percept["obstacle_ahead"]:
        return "navigate_around"
    if percept["sample_detected"]:
        return "collect_sample"
    return "move_forward"
```

```
    def act(self, action):
        if action == "collect_sample":
            self.samples_collected += 1
            self.battery -= 5
        elif action == "move_forward":
            self.position = (self.position[0] + 1, self.position[1])
            self.battery -= 3
        elif action == "navigate_around":
            self.battery -= 4
        elif action == "return_to_charge":
            self.battery = 100
        print(f"Action: {action} | Position: {self.position} | Battery: {self.battery} | Samples: {self.samples_collected}")

    def run(self, steps=10):
        for _ in range(steps):
            percept = self.perceive()
            self.update_model(percept)
            action = self.decide_action(percept)
            self.act(action)

rover = MarsRoverAgent()
rover.run(10)
```

3. 8 Queens

```
def is_safe(board, row, col, n):
    for i in range(row):
        if board[i] == col or \
           abs(board[i] - col) == abs(i - row):
            return False
    return True


def solve_n_queens(board, row, n, solutions):
    if row == n:
        solutions.append(board[:])
        return
    for col in range(n):
        if is_safe(board, row, col, n):
            board[row] = col
            solve_n_queens(board, row + 1, n, solutions)
            board[row] = -1  # backtrack


def print_board(solution, n):
    for row in range(n):
        line = ""
        for col in range(n):
            line += "Q " if solution[row] == col else ". "
        print(line)
    print()
```

```
n = 8
board = [-1] * n
solutions = []
solve_n_queens(board, 0, n, solutions)

print(f"Total solutions found: {len(solutions)}")
print("\nFirst solution:")
print_board(solutions[0], n)
```

4. OLA Cab Booking

```
class CabBookingAgent:
    def __init__(self):
        self.cab_options = {
            "micro": {"base_fare": 40, "per_km": 8, "wait_time": 5},
            "mini": {"base_fare": 50, "per_km": 10, "wait_time": 4},
            "sedan": {"base_fare": 80, "per_km": 14, "wait_time": 6},
            "shared": {"base_fare": 25, "per_km": 5, "wait_time": 10},
            "prime": {"base_fare": 100, "per_km": 18, "wait_time": 3},
        }

    def calculate_fare(self, cab_type, distance_km):
        cab = self.cab_options[cab_type]
        return cab["base_fare"] + cab["per_km"] * distance_km
```

```
    def evaluate(self, cab_type, distance_km, priority="cost"):
        fare = self.calculate_fare(cab_type, distance_km)
        wait = self.cab_options[cab_type]["wait_time"]

        if priority == "cost":
            return -fare  # lower fare = better score
        elif priority == "time":
            return -wait  # lower wait = better score
        else:
            return -(fare * 0.6 + wait * 0.4)  # balanced

    def book_best_cab(self, distance_km, priority="cost"):
        best_cab, best_score = None, float("-inf")
        for cab_type in self.cab_options:
            score = self.evaluate(cab_type, distance_km, priority)
            if score > best_score:
                best_score = score
                best_cab = cab_type

        fare = self.calculate_fare(best_cab, distance_km)
        print(f"Recommended cab: {best_cab.upper()} | Estimated Fare:
Rs.{fare} | ETA: {self.cab_options[best_cab]['wait_time']} min")
        return best_cab

# Example run
```

```
agent = CabBookingAgent()
agent.book_best_cab(distance_km=12, priority="cost")
agent.book_best_cab(distance_km=12, priority="time")
```

5. Logistics Network

```
import heapq

def ucs(graph, start, goal):
    frontier = [(0, start, [start])]  # (cost, node, path)
    visited = {}

    while frontier:
        cost, node, path = heapq.heappop(frontier)

        if node == goal:
            return path, cost

        if node in visited and visited[node] <= cost:
            continue
        visited[node] = cost

        for neighbor, weight in graph.get(node, []):
            new_cost = cost + weight
            if neighbor not in visited or new_cost < visited.get(neighbor,
float('inf')):
```

```
                heapq.heappush(frontier, (new_cost, neighbor, path +
[neighbor]))


    return None, float('inf')


# Graph from the figure
graph = {
    "S": [("A", 1), ("G", 12)],
    "A": [("B", 3), ("C", 1)],
    "B": [("D", 3)],
    "C": [("D", 1), ("G", 2)],
    "D": [("G", 3)],
    "G": []
}


path, cost = ucs(graph, "S", "G")
print(f"Least-cost path: {' -> '.join(path)}")
print(f"Minimum cost: {cost}")
```